

Software Project Management – IS212

Agile Estimation & Scrum Events

Announcements

Announcements – Project Groups & Instructions

- **Project Groups:** please register your teams in eLearn today
- The **Project Instructions** will be released **Week 3 Friday at 6pm**
- This will clarify the **scope for the “first release”** (i.e., due Week 12) as well as how we would like you to document your scrum process
 - *Please read it carefully and let us know if you have any questions*
 - *Be sure to start your first sprint in Week 4 so you can finish on time*

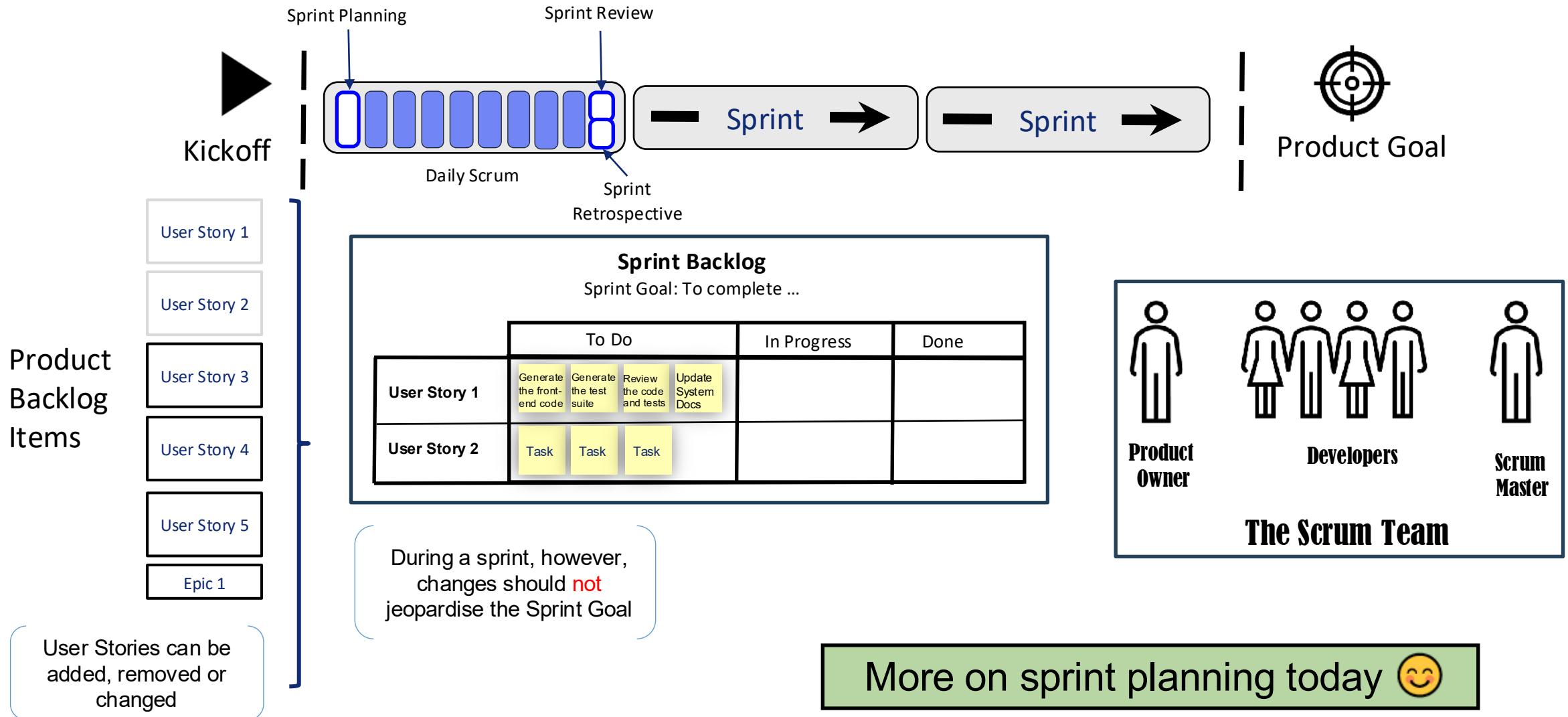
What should you do with your product backlog after the scope of the first release is clarified?

Recap

Week 2 Recap – User Stories

- In the **user story template** "*As a... I want... so that...*", what do the three parts capture?
- What are the **three Cs** of a user story, and which one is arguably "the whole point"?
- What must **acceptance criteria** be, and what do they define for a story?
- What does **INVEST** stand for? How has AI shifted the "weights"?
- What is an **epic**, and what is it used for on the product backlog?
- How can **AI be used effectively** in product backlog management?

Week 2 Recap – User Stories



Estimation

Estimation & Scrum Events

Objectives

On completing this class, you will be able to:

- Explain why software estimation is **uncertain**, and why **relative estimation** can help
- Apply **story points** & **planning poker** as a classic approach
- Compare **velocity-based** and **flow-based** planning (throughput, cycle time)
- Use estimates and evidence to **support** – not replace – team judgement
- Explain the purpose and conduct of the **Scrum events**



Outline

- Planning under uncertainty: story points, velocity & flow metrics
- Planning poker, probabilistic forecasting & burndown charts
- Scrum events as feedback loops: planning, daily scrum, review & retrospective

Do Agile Teams Have to Estimate?

- **No.** Scrum requires no specific technique – not story points, not velocity, not planning poker
- Teams estimate **when doing so helps them make a decision** – about scope, sequencing, staffing, or delivery
- The useful part is often the **conversation** and the uncertainty it reveals – not the number itself
- **Today:** we'll learn a classic approach (**story points**) and discuss how AI is changing what effort looks like and how reliably we can forecast it
- We'll also look at alternative techniques based on **actual flow**

Estimation in Software Development

- An **estimate** is not a promise – it is a **decision aid** under uncertainty
 - *It is typically a range, probability, or relative comparison*
 - **NOT** a precise number
- It may inform decisions about **scope, effort, cost, tokens, ...**
- It is based on **incomplete information, judgement, and evidence**
- **Estimation** is the **process** of reducing uncertainty enough to make a useful decision



"This project requires 50 person-months of effort"



In most **single-point estimates**, the level of certainty is **not** "100%"



...50 person-months later

basis for competitive contracts (money, time, ...)

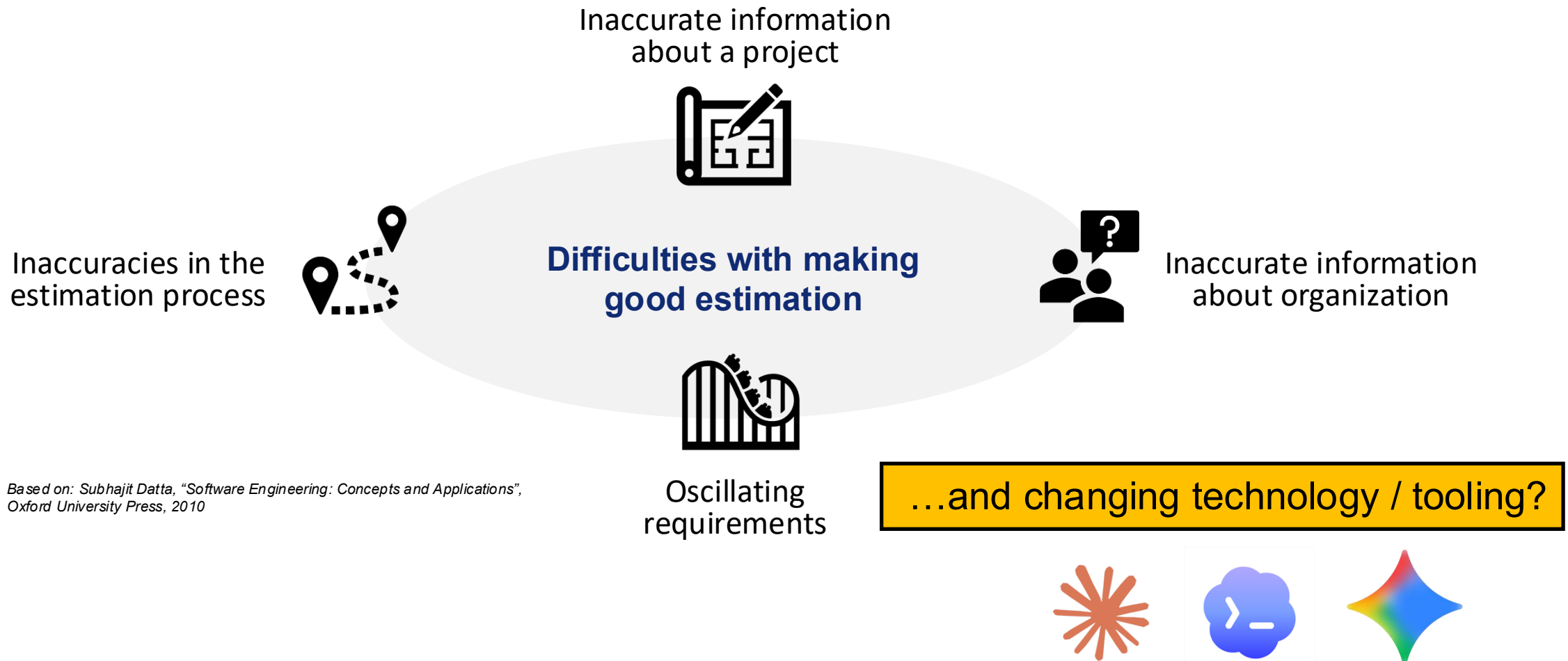
efficient resource planning (headcount, tokens, ...)

Why are (good) software estimates **important?**

determine scope/timing of releases

basis for comparing “actual” vs. “planned” work done

Why is Estimation Difficult in Software Development?



Based on: Subhajit Datta, "Software Engineering: Concepts and Applications", Oxford University Press, 2010

What Leads to Better Estimates?

- Clarity of the goal
- Clarity of the steps towards the goal
- Clarity of duration of each step. **How?**
 - Experience with *similar projects* in the past
 - Experience with *required technologies* and tools
 - Experience working with the *same teams* and customers
 - *Knowledge* of existing dependencies across teams or projects
- Anticipation of a change and *accommodating the unknown*



"[...] estimating and planning processes
must **themselves be agile**.

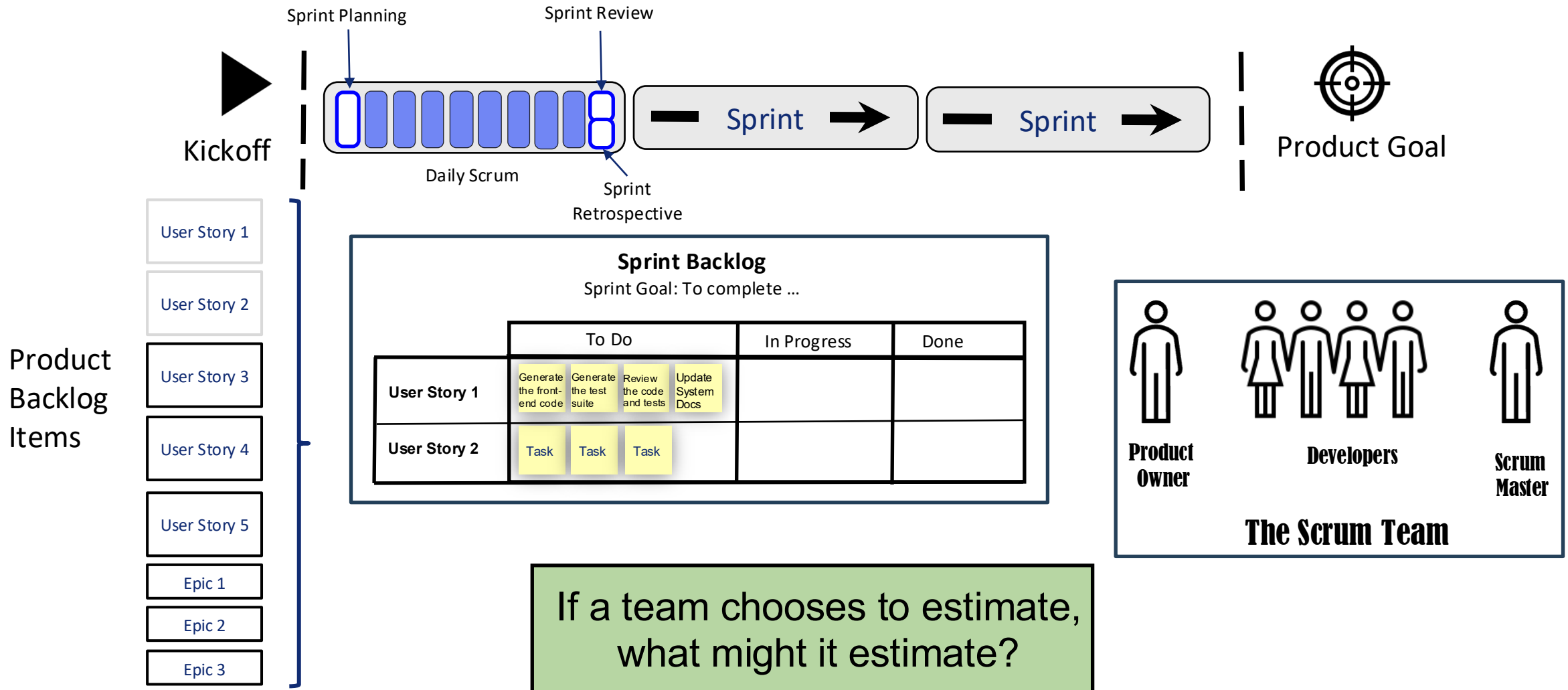
Without agile estimating and planning, we
cannot have agile projects."

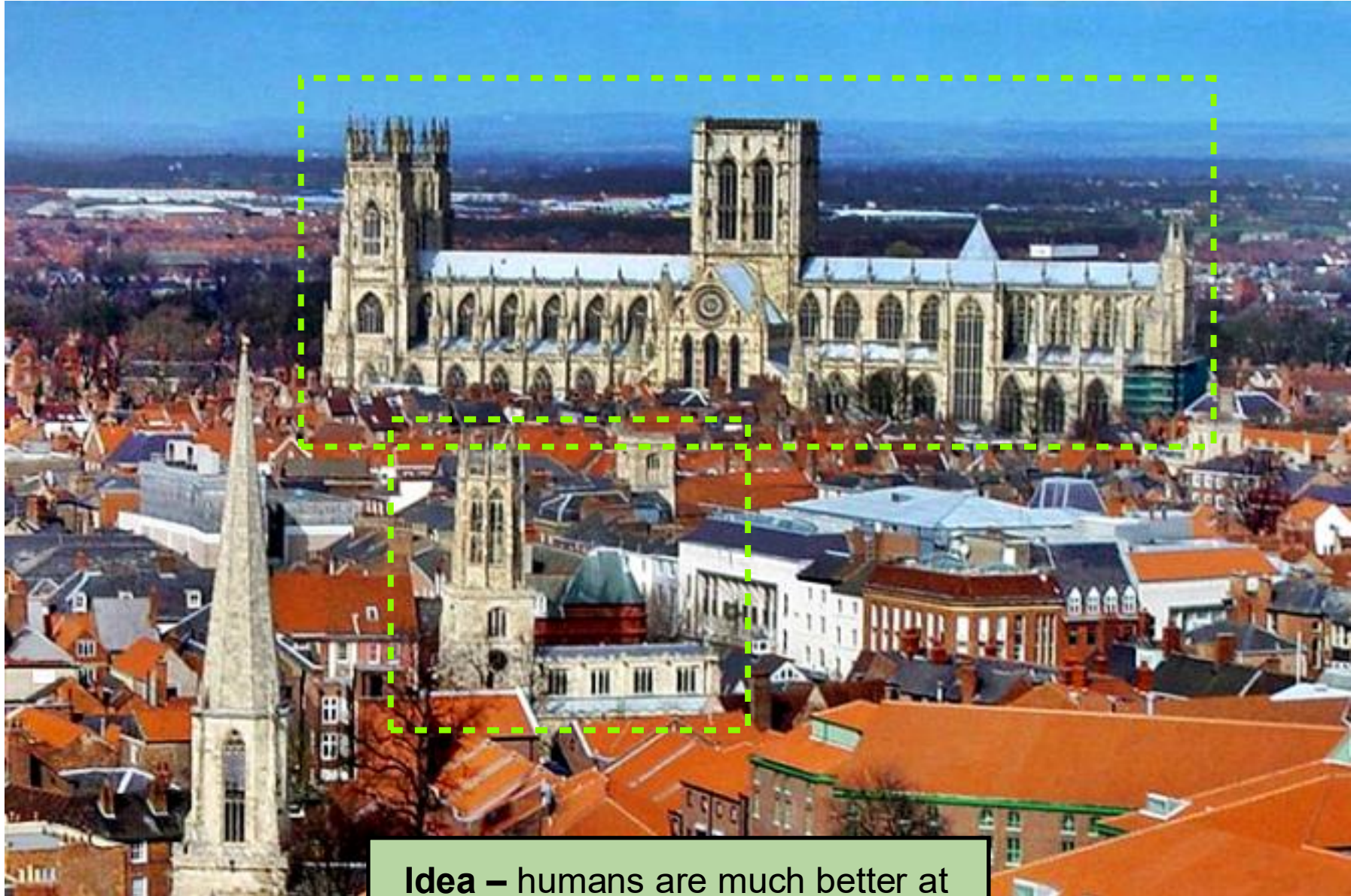
Mike Cohn

Agile Estimation



Estimation in Scrum





Idea – humans are much better at making **relative** estimates

Agile Estimates – Story Points

- **Story points** are one way of expressing the **relative size of work**
 - They may reflect **effort**, **complexity**, **uncertainty**, and **coordination**
- They are **NOT** an absolute measurement like no. of hours
- They are **relative buckets**, not precise measurements
 - A “5 point” item is considered substantially larger than a 2-point item
 - It does **NOT** mean exactly 2.5 times the work
- Teams develop shared reference points from previously completed work (e.g. a “1-pointer”, a “2-pointer”, ...)
- AI changes what “**effort**” means
 - Code generation is **cheap**; validation, integration, testing, review, and uncertainty now **dominate**



How to Start?

- **Step #1: Choose a reference story**

- Pick a **small, well-understood story**
- Assign it a convenient value, e.g. **1 or 2 story points**
- ...don't fret 😎



- **Step #2: Estimate relatively**

- For each new story, ask: *“is it smaller or larger than the reference?”*
- *“How much more complexity, uncertainty, coordination, or validation is involved?”*
- Place it into the **closest story-point bucket**
- ...don't fret 😎

- **Step #3 & beyond: Calibrate as you learn**

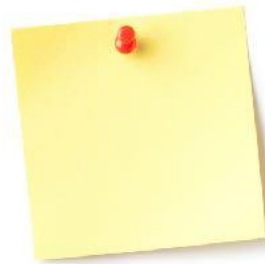
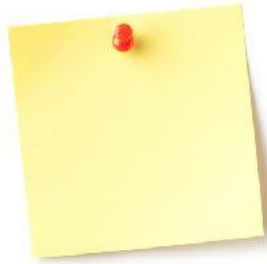
- Use **completed stories** as **better reference points** over time
- Revisit your intuition when reality differs from the estimate
- ...don't fret 😎

Story Point Values

- 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, ... ? ❌
 - Too fine-grained
 - Larger stories have **more uncertainties** – harder to estimate accurately
 - **Waste of time** to argue whether a story is 14 pts vs. 15 pts (etc.)
 - We rather want to **put the story in the 'right ballpark'** and move on
- Studies show that humans perceive **differences between smaller numbers more precisely** than differences between larger numbers
 - E.g. the difference between 1 vs. 3pts is easier to perceive than, say, 15 vs. 16pts
- Consequently, many scrum teams estimate using **only** the values of a non-linear scale, e.g. **modified fibonacci**:
 - 1, 2, 3, 5, 8, 13, 20, 40, 100

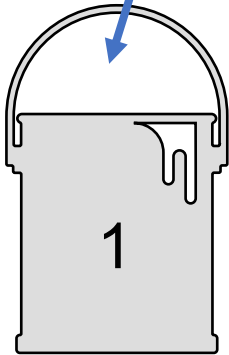
Why the big jumps?

"smallest" story

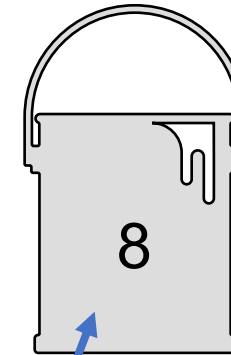
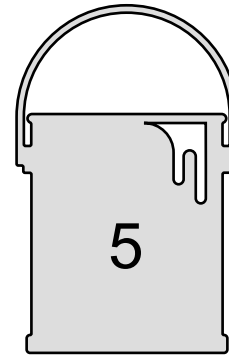
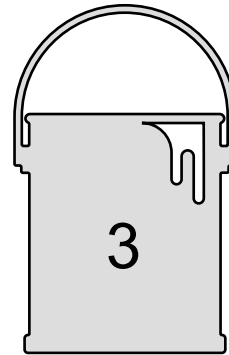
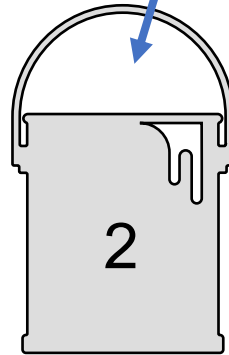


*more complexity and
uncertainty than the
reference story*

assign 1 point



assign 2 points



assign 8 points

*What if a story is larger
than 8 points?*

It likely needs more conversation
(and perhaps) splitting



Feels closer to
8 than 5
...?

Alternative: T-shirt Sizes (~5 Buckets)



Estimate: ...

User Story

*As a **student**, I want to **view my profile** so that I can check that my **personal and academic details are correct***

Acceptance Criteria

Ensure the student can:

- * View their name, email address, year of study, and campus ID*
- * View only their own profile*
- * See a clear message if their profile cannot be loaded*

Estimate: ...

User Story

*As a **student**, I want to **view forthcoming courses** so that I can **identify courses I am eligible to take***

Acceptance Criteria

Ensure the student can:

- * View course title, code, description, and pre-requisites*
- * View only forthcoming courses*
- * View only courses they have not already taken*
- * Filter to courses for which they satisfy all pre-requisites*
- * See a clear message if no eligible courses are available*

Estimate: ...

User Story

*As a **faculty**, I want to **update the pre-requisites** for a course I manage so that **students are shown the correct eligibility requirements***

Acceptance Criteria

Ensure the faculty can:

- * Select only courses they manage*
- * View the current pre-requisites for the selected course*
- * Add an existing course as a pre-requisite*
- * Remove an existing pre-requisite*
- * Prevent invalid pre-requisites (the course itself; a mutex course; a course that would create a circular pre-req dependency)*
- * Save the changes*

Common Estimation Techniques

- **Expert judgement**
 - Use experience, domain knowledge, technical knowledge, and evidence
- **Analogy**
 - Compare new work with similar work completed previously
 - Place it into an appropriate relative-size bucket
- **Decomposition**
 - Break large or uncertain work into smaller pieces
 - Smaller pieces are generally easier to understand and estimate
- Alternatively, **collaborative estimation** techniques like **planning poker**  facilitate the **conversations** that help surface different assumptions



Estimation using planning poker



Planning Poker – Steps



1. Each estimator given a deck of cards: 1, 2, 3, 5, 8, 13, 20, 40, and 100.
2. For each user story to be estimated, a moderator reads the description and answers any questions that the estimators have.
3. Each estimator privately selects a card representing his or her estimate.
 - Cards are not shown until each estimator has made a selection.
 - At that time, all cards are simultaneously turned over and shown so that all participants can see each estimate.
 - Typically, the people doing the work estimate it; the Product Owner provides context and answers questions but does not estimate.
4. If estimates differ, the high and low estimators explain their estimates and discuss them among the team.
5. After the discussion, each estimator re-estimates by selecting a card.
6. Repeat as needed until the team has enough shared understanding to agree an estimate

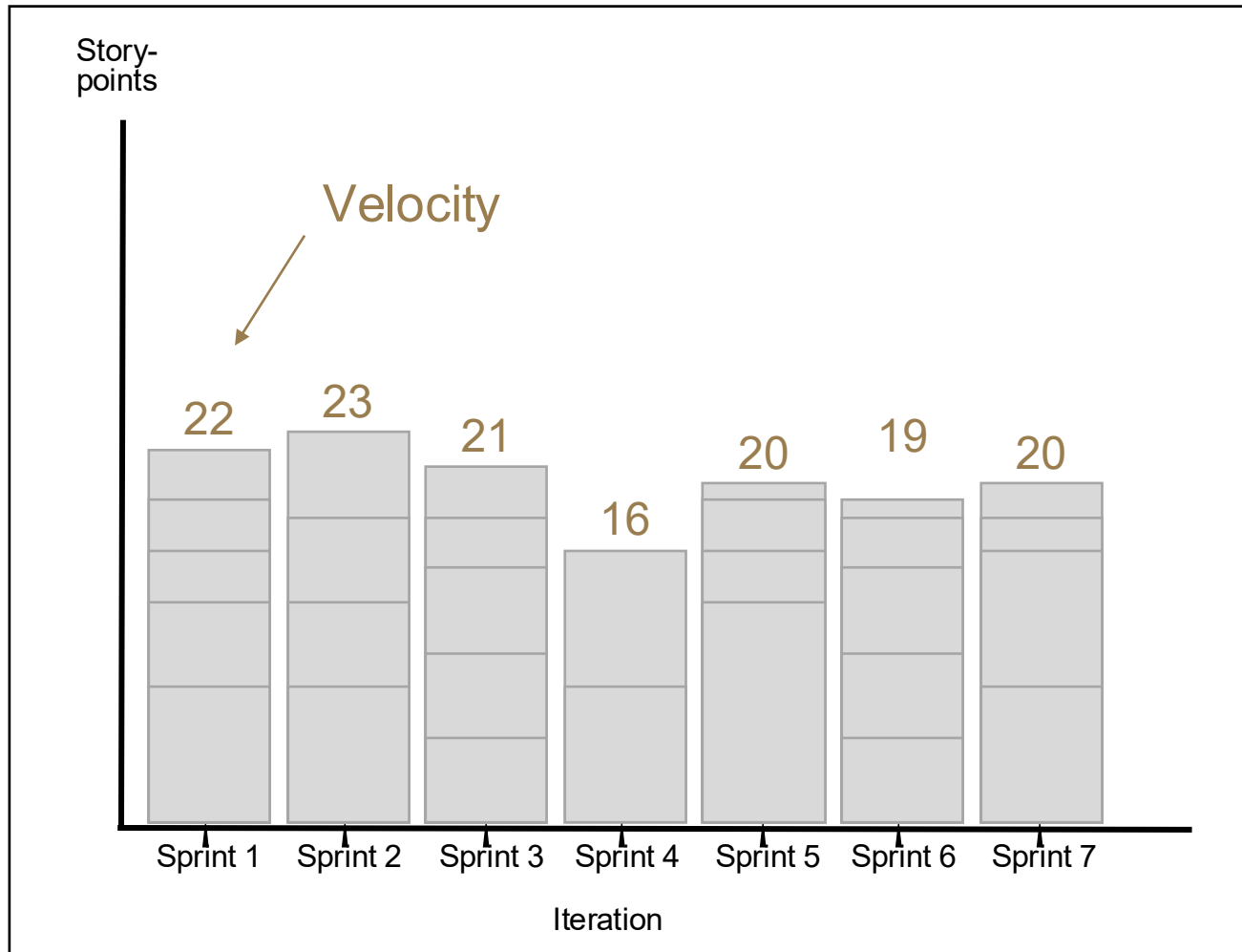
Velocity & Burndown Charts

Velocity – *how much work are we completing?*

- **Velocity** records how many story points of completed work a team delivered in a Sprint
- Measured by **summing the no. points** of the user stories fully completed in the last sprint
- Example – in two sprints the team completed...
 - **Sprint 1:** Three 5-point stories **← velocity 15**
 - **Sprint 2:** Three 3-pt stories, three 2-pt stories **← velocity 15**
- Recent velocity suggests that ~15 points may be a reasonable planning reference for the next sprint (points *not* number of stories!)
- **Caution:** velocity is a planning aid, **not** a productivity score. Don't compare it across teams, don't target ever-higher velocity, and treat past velocity cautiously when the team or technology changes (e.g. **adopting agentic coding**)



Measuring Velocity



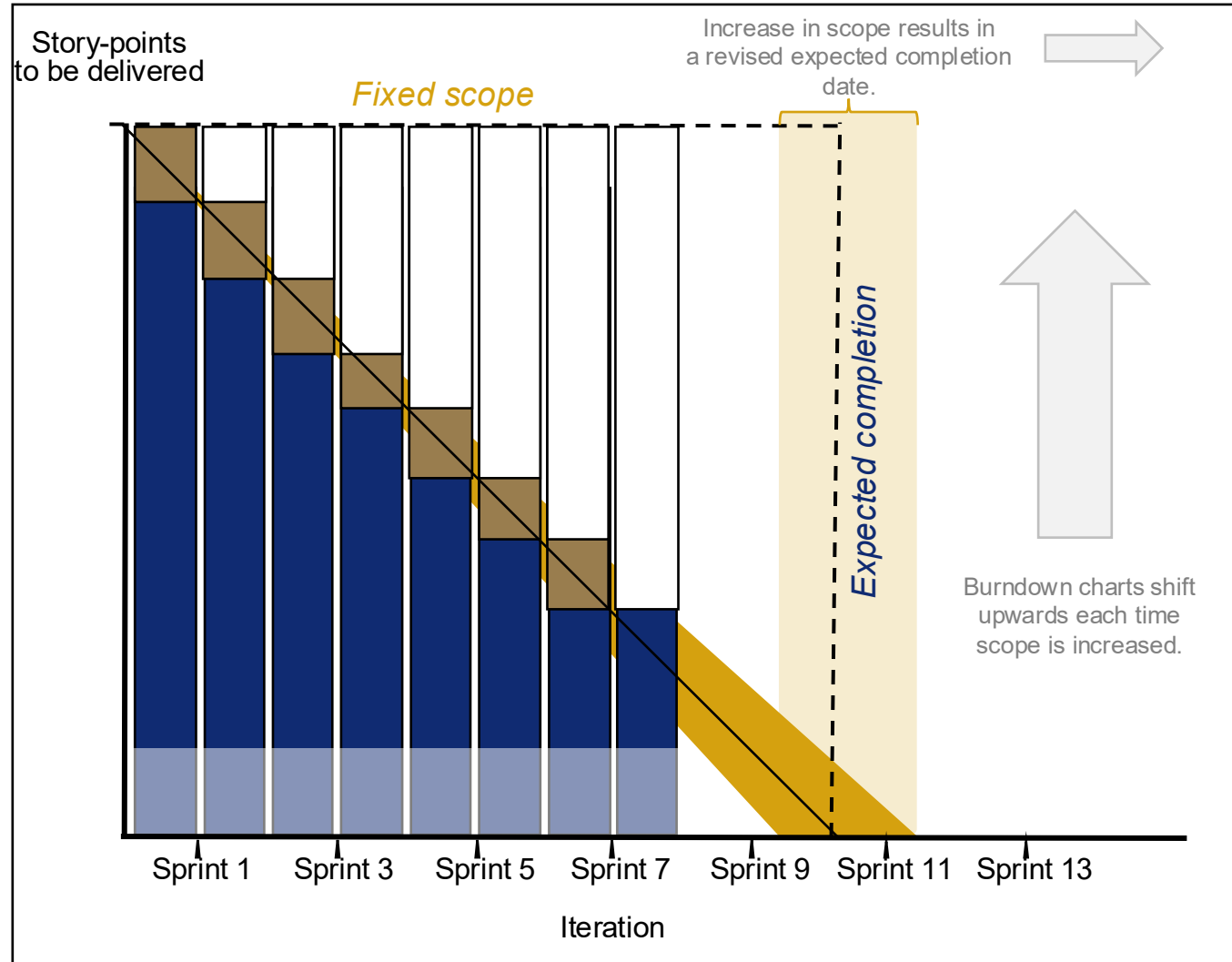
Historical velocity can help teams judge how many story points they can reasonably tackle in the next sprint.

For the first few sprints, teams have to make assumptions based on their past work and experience.

Separation of Effort and Duration

- We **estimate the effort** of user stories (using a relative approach)
- Historical velocity can inform a **rough forecast** of how many Sprints remaining work may require
 - i.e. we are **NOT** directly estimating duration
- If a team's velocity changes, we can re-calculate the completion of the release / project **without re-estimating** any user stories
- **Example:**
 - Suppose a team estimates a release to include **200** story points of work
 - This initially leads them to believe that they can complete it in **10** sprints, if they work with their usual velocity of **20** story points per sprint
 - Once the project begins, they observe velocity of **16** story points. **Without re-estimating any work**, they can now revise the forecast to about **13** sprints (rather than **10**)

Burndown Charts – *tracking the work remaining*



Burndown charts show how the cumulative story points delivered reduce the **remaining effort** to deliver the project.

They can help teams notice whether progress or scope is diverging from expectations

How AI Assistants Affect Estimation

- AI coding assistants (Copilot, Claude Code, Cursor) change the effort profile of work **unevenly**
 - Boilerplate, CRUD, and scaffolding can get much faster
 - Novel design, debugging, and integration into a large codebase might not – and reviewing AI output adds effort
- **Consequence 1** – historical velocity may no longer predict future velocity, so teams must **re-baseline**
- **Consequence 2** – reference stories need recalibrating (a former “5” might now be a “2” ... or not)
- **Estimate the whole story:** writing code is now rarely the bottleneck – understanding, review, and integration are
- **Aside:** AI also adds costs beyond developer time (e.g. token / inference cost) – useful for budgeting, but don't confuse it with delivery forecasting

Developers expected...

- Predicted AI would make them ~24% faster
- Even afterwards, they still believed they had been ~20% faster
- By 2025, ~84% of developers were using AI tools

Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity

Joel Becker*, Nate Rush*, Beth Barnes, David Rein

Model Evaluation & Threat Research (METR)

<https://arxiv.org/abs/2507.09089>

Reality (METR RCT, 2025)

- A randomised controlled trial: 16 experienced devs on their own mature repos (1M+ lines)
- **19% slower** with early-2025 AI tools; Trust in AI accuracy had fallen to ~33%
- **A near-total gap between felt and measured productivity**
- **2026 update** – METR's follow-up suggests newer AI tools may now improve productivity, but selection effects make the size of that benefit difficult to measure reliably

Conclusion? AI capability is changing fast and intuition about productivity is unreliable. Measure instead **what actually happens**

An Alternative: Planning from Flow Metrics

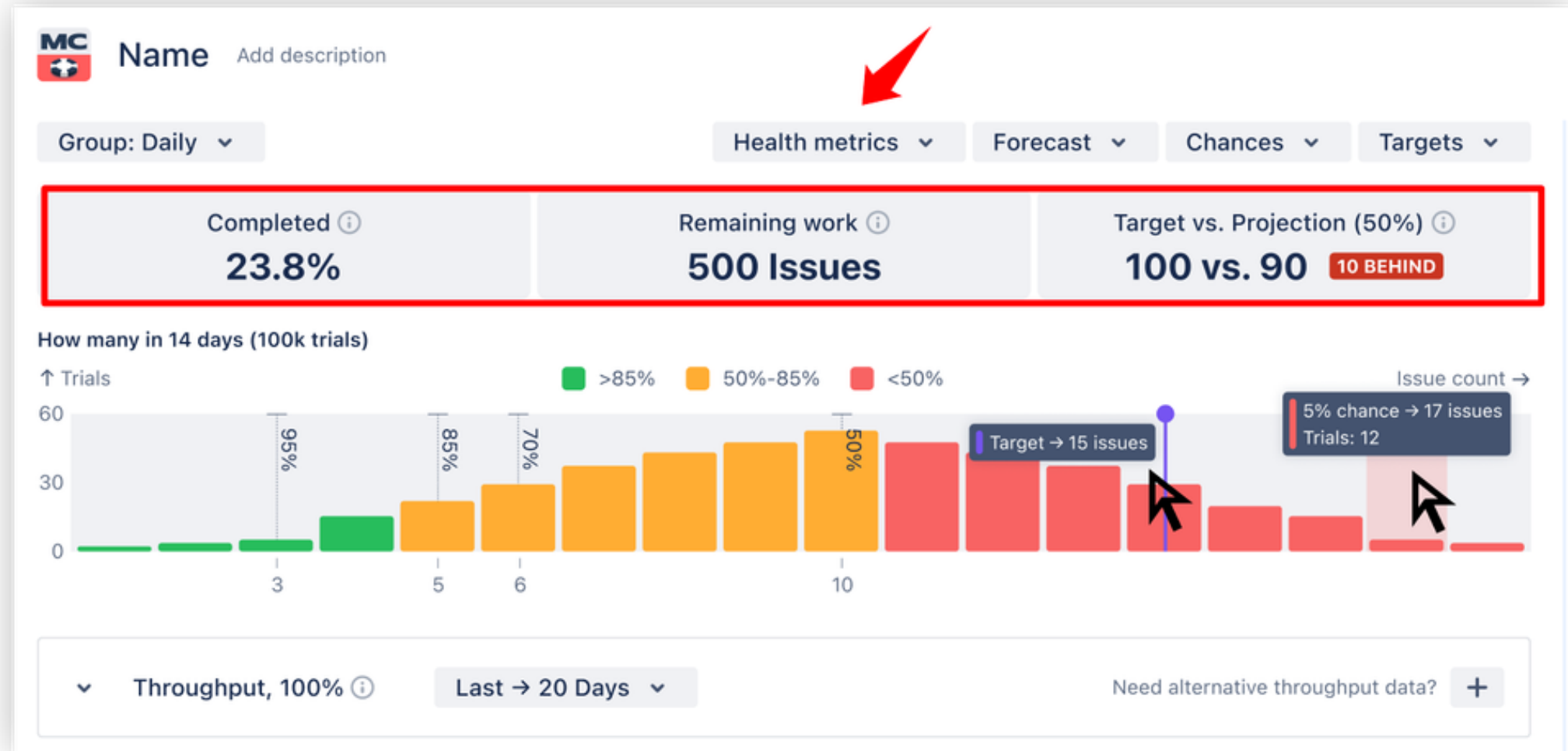
- **Rather than estimating the size of future work**, teams can learn from what actually happened
- **Throughput** – the number of work items completed per sprint (or per week)
- **Cycle time** – how long an item takes to go from “In Progress” to “Done”
- **WIP** (work-in-progress) – how many items are in flight at once; limiting WIP tends to shorten cycle time
- **Story points ask:** “How big do we **think** this is?”
Flow metrics ask: “How does work **actually** move through this team?”

***Key idea:** throughput and cycle time need no estimation. They measure what actually happened*

Probabilistic Forecasting based on Flow Metrics

- Recall the false certainty of a **single-point estimate** (“50 person-months”) – the real world is rarely that precise
- **Monte Carlo forecasting** replaces the single number with a probability distribution
 - It repeatedly samples the team’s historical throughput to simulate thousands of possible futures
 - **Flow metrics** measure what already happened; Monte Carlo turns them into a forward forecast
- The output is a **probability**, e.g. “**85% chance of finishing these 30 items by 5 May**” – not a false-precision date
- **The point:** good forecasting **expresses uncertainty**, rather than hiding it behind a single date
- **Jira** apps/tools now offer Monte Carlo forecasting from throughput

Probabilistic Forecasting based on Flow Metrics



<https://community.atlassian.com/forums/App-Central-articles/See-delivery-outcomes-with-confidence-Agile-Monte-Carlo-Charts-1/ba-p/3144668>

So... Should We Still Estimate?

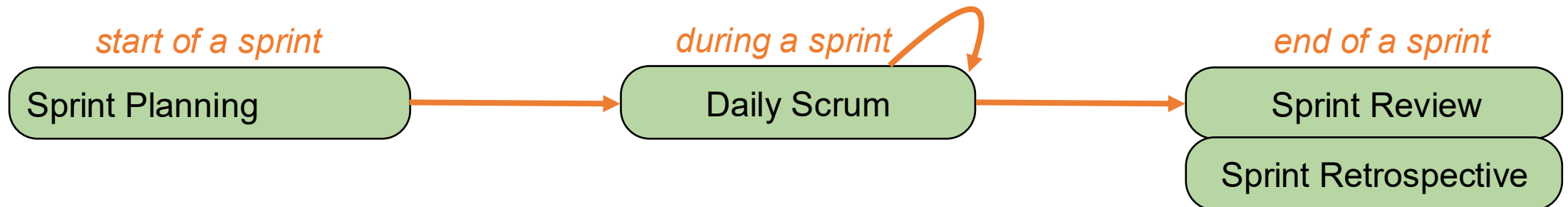
- **Yes – but less reflexively.** Estimate when the uncertainty is worth reducing, not because Scrum requires a number (**it doesn't**)
- The value is usually in the **conversation** the estimate provokes – exposing disagreement, spotting oversized stories, comparing scopes – not whether it's a 3 or a 5
- Estimation is a **planning & conversation tool**, not a measure of effort or developer **productivity**
- Scrum mandates **no technique** – story points, coarse S/M/L, counting similarly-small stories, or historical throughput / cycle-time forecasting are all valid

Scrum Events

Recap – Overview of Scrum

- **Sprints** are fixed-length (e.g. 2, 3, or 4 week) iterations that aim to achieve an agreed **sprint goal**, e.g. a particular software increment
 - For the IS212 project, we recommend choosing a sprint length of **1 or 2 weeks**
 - The **Sprint Goal** should remain stable during the sprint; scope may be renegotiated without jeopardising it
- Sprints are lightly managed using “time-boxed” scrum events (also called ‘ceremonies’ or ‘meetings’)

Why is time-boxing important?



Scrum Events are Feedback Loops

- Every Scrum event exists to answer a question – they are the team's **feedback loops**
- **Sprint Planning** → What valuable outcome should we pursue next?
- **Daily Scrum** → Are we still on course for the Sprint Goal?
- **Sprint Review** → Did we build the right thing? What did we learn from stakeholders?
- **Sprint Retrospective** → How should we change the way we work?
- **AI** can cut the admin cost of these events but it doesn't replace the feedback loops they create

Sprint Planning

- **Sprint planning** takes place at the start of a sprint, time-boxed to a **maximum of 8 hours** for a one-month sprint
 - Shorter sprints usually require less time
- **Purpose:**
 - Define **why** the Sprint is valuable → Sprint Goal
 - Decide **what** can be Done → selected Product Backlog Items
 - Plan **how** the work will be accomplished
- **Attended by the whole scrum team**

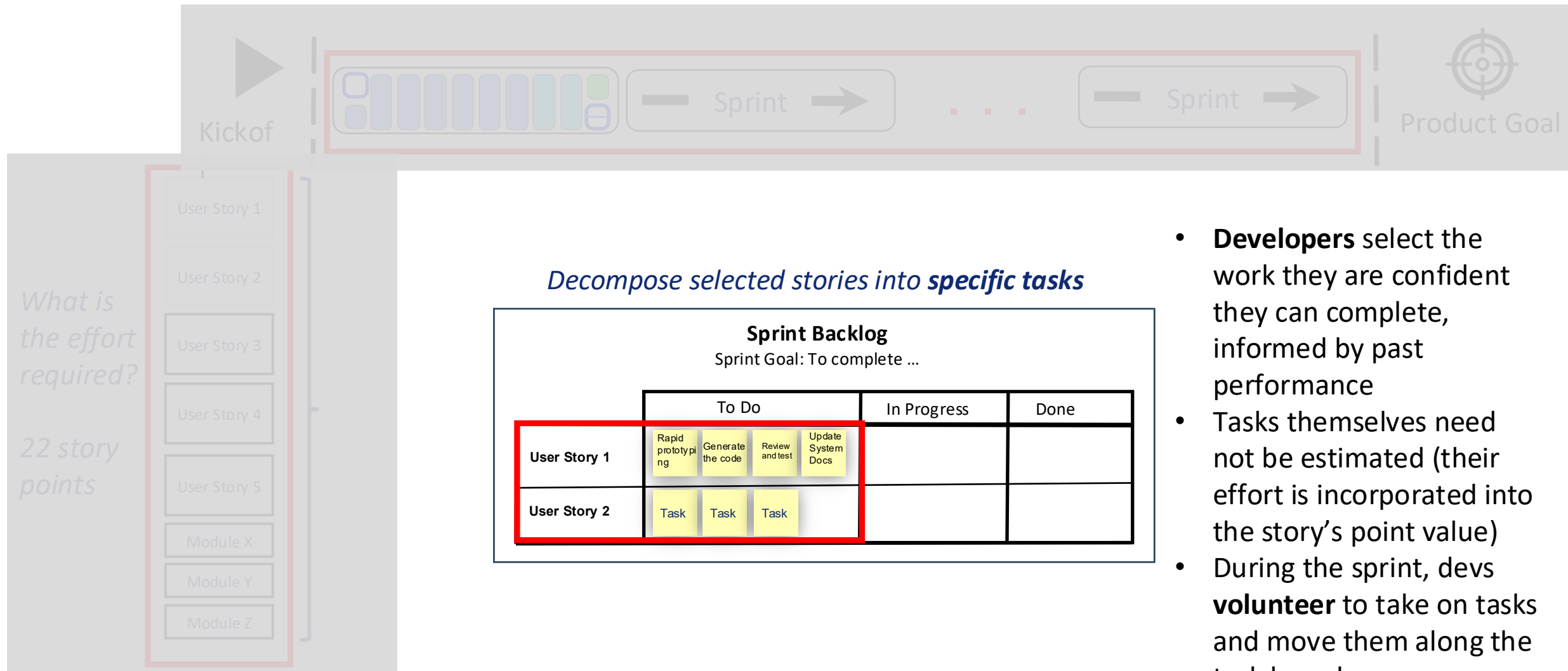


<https://www.scrum.org/resources/what-is-sprint-planning>

Sprint Planning – Activities

- **Review / clarify Product Backlog Items** – Product Owner brings the **highest-priority items** and answers questions about value, scope, and requirements
- **Clarify / estimate where needed** – discuss items that need further clarification, splitting, or estimation 
- **Form the Sprint Backlog** – Developers select the work they believe they can accomplish, informed by **past performance**, **current capacity**, **dependencies**, and **uncertainty**, while working toward a coherent Sprint Goal
 - *Create an initial plan for delivering the selected items*
 - *Decompose items into **specific tasks** where useful*

Sprint Planning – Tasks



Sprint Planning Benefits

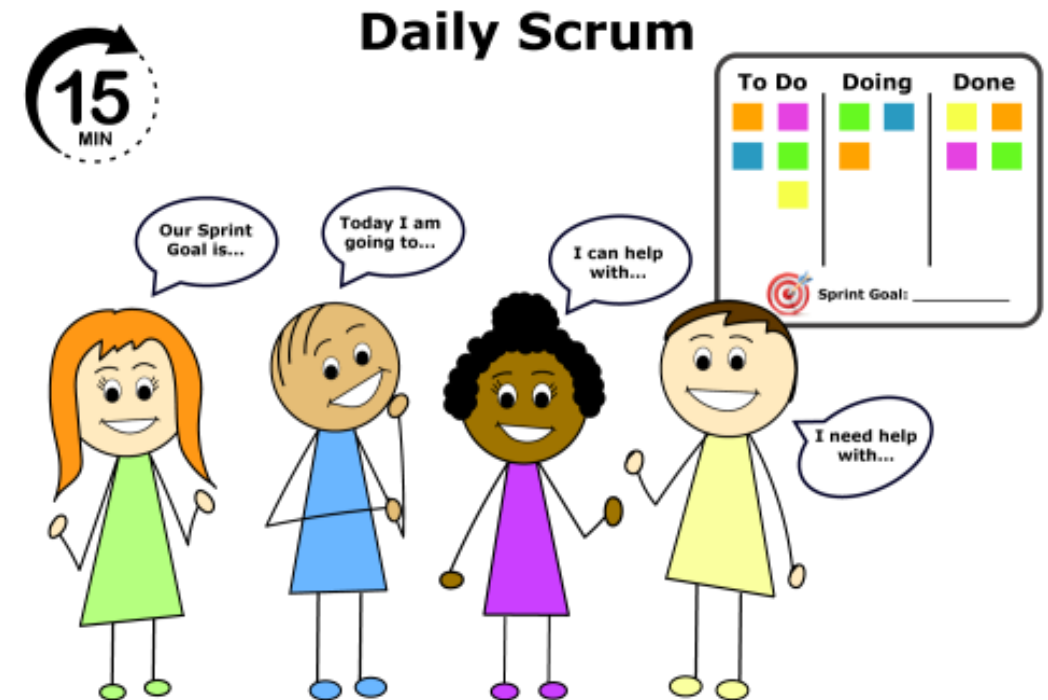
- **Aligns** the team on what needs to be achieved in the next sprint
- **Supports sustained workload** – the team determines a realistic goal for the sprint and selects an achievable amount of work based on **capacity and past performance**
- **Improves shared understanding** – creates space to discuss priorities, dependencies, technical concerns, and assumptions

Sprint Planning Challenges

- **Overcommitment** – especially when the team has little historical evidence yet
- **Overloading the Sprint** – selecting **too much work** or ignoring known capacity constraints
- **Over-planning** – spending too much time **trying to predict details** that may change once work begins
- **Team dynamics** – differences in experience, communication, or assumptions can hinder planning

Daily Scrum (aka Daily Stand-Up)

- The **daily scrum** takes place each day, time-boxed to **15 mins**
- **Purpose:**
 - **Inspect progress toward the Sprint Goal**
 - Adapt the **Sprint Backlog / plan** for the next day of work
 - Identify **obstacles** (“blockers”) and coordination needs
 - **NOT** a status meeting for the scrum master or managers
- **Led by the Developers – supported by the scrum master**



<https://www.scrum.org/resources/what-is-a-daily-scrum>

Daily Stand-up (aka Daily Scrum)

- There are many different ways to conduct a stand-up
 - Many different tools (e.g. **Miro**), bots, etc. to support teams too
 - **Async stand-ups** are common for distributed or part-time teams – written updates via a Slack bot (e.g. Geekbot) instead of a live meeting
 - **One common format** – each member answers:
 - What did I do yesterday?
 - What will I do today?
 - Are there any **impediments** in my way?

What kind of impediments might be encountered?

 - **A more goal-focused set:** Where are we vs the Sprint Goal? What changed? What should we adapt today?
- **Important:** this is to synchronise and assist, **not** to blame!

Stand-up Benefits

- Enhances **communication** and collaboration
- Supports **connection and coordination** in distributed/hybrid teams
- Keeps the team **focused** and aligned on goals
- **Surfaces blockers** and coordination needs early
- Helpful beyond scrum – many waterfall teams do stand-ups!

Stand-up Challenges

- **Uneven participation** – some voices dominate while others disengage
- Deep problem-solving can consume the time-box; take detailed discussions **offline** with the relevant people
- **Developers** keep it focused and within 15 minutes; the Scrum Master coaches the team if needed

Sprint Review

- The **sprint review** takes place at the end of a sprint, time-boxed to **4 hours** for a one-month sprint
 - Shorter sprints usually need shorter reviews
- **Purpose:**
 - Inspect the **software increment** and **sprint outcomes**
 - Discuss progress towards the overall **product goal**
 - Gather **stakeholder feedback** and consider changes/opportunities
 - Adapt the **product backlog** as necessary
- Attended by the **scrum team** plus key stakeholders invited by the product owner



<https://www.scrum.org/resources/what-is-a-sprint-review>

Sprint Review

- **Key Activity:** inspect the **Sprint outcome** and discuss progress toward the **Sprint Goal / Product Goal**
- The Scrum Team reviews what was **Done** and what was **not Done**

What does “Done” mean?
- Inspect the **working software increment** with stakeholders and answer questions
- Feedback (e.g. required enhancements) may lead to **Product Backlog changes** or new items
- Items that are not Done do **not** count towards velocity and are reconsidered in the Product Backlog – they may be re-planned, changed, split, or dropped

Sprint Review Benefits

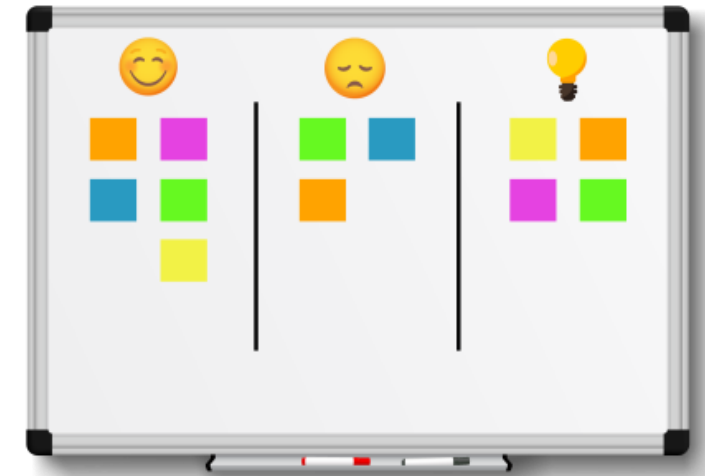
- Early stakeholder **feedback and learning**
- **Transparency** through inspecting actual outcomes
- Keeps future work aligned with the **Product Goal and stakeholder needs**
- Recognises progress and useful outcomes

Sprint Review Challenges

- **Unclear Definition of Done** can cause disagreement about what is inspectable (**see Week 4!**)
- Feedback **≠** commitment – new requests still need Product Backlog decisions
- Review **becoming a demo/status presentation** rather than collaborative inspection
- **Low stakeholder engagement** limits useful feedback and adaptation

Sprint Retrospective

- The **sprint retrospective** concludes the sprint, time-boxed to maximum of **3 hours** for a one-month sprint
 - Shorter sprints will usually have shorter retros
- **Purpose:**
 - **Inspect** how the last sprint went with respect to people, interactions, processes, tools, and the Definition of Done
 - Identify what **went well**, **what problems occurred**, and how they were or were not solved
 - Select the **most useful improvements** to increase quality and effectiveness
 - Put those improvements **into practice** as soon as possible
- Attended by the whole Scrum Team; the Scrum Master helps ensure the event is productive and effective



<https://www.scrum.org/resources/what-is-a-sprint-retrospective>

Sprint Retrospective

- The scrum team can use **different retrospective techniques** to structure reflection and discussion
- For example, using:
 - **4Ls**: Liked, Learned, Lacked, Longed For
 - **Start-Stop-Continue**
- Based on the insights, the team **collaboratively identifies one or more concrete improvements** to try in the **next sprint**
 - They should be actionable, realistic, and within the team's control
- Retrospectives can also work well remotely using shared whiteboards or collaboration tools (e.g. **Miro**, EasyRetro)



Sprint Retro Benefits

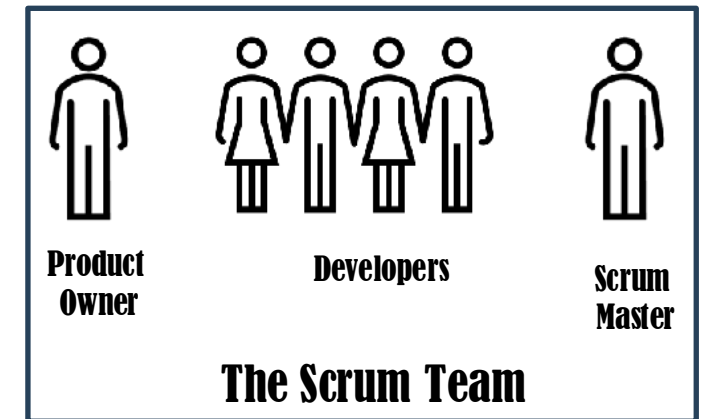
- A structured opportunity to **foster continuous improvement**
- Encourages **reflection**, learning, and proactive improvement
- Creates space for team members to **raise concerns, share ideas, and influence** how the team works
- Can strengthen trust, team cohesion, and shared ownership

Sprint Retro Challenges

- **Negativity / blame** – retros can become complaint sessions unless discussion stays **constructive and evidence-based**
- Action without follow-through – improvement ideas **need owners or a clear plan** and should be revisited
- Low psychological safety – people may **withhold concerns** if they fear judgement or consequences
- Retro fatigue – repeating the same format or **discussing the same unresolved issues** can reduce engagement

Backlog Refinement (aka 'Grooming')

- **Not a formal Scrum event** – backlog refinement is an ongoing activity, undertaken as needed
 - Can occur at **any** time
 - Usually focuses on **upcoming product backlog items** while the current sprint continues
- **What's it about?**
 - Clarify and order Product Backlog Items
 - Add detail as needed so upcoming items are sufficiently understood
 - Split, combine, or reshape items
 - Identify dependencies, assumptions, and uncertainty
 - Estimate where useful
 - A useful heuristic: keep the backlog **DEEP**
- The **Product Owner is accountable** for product backlog management and typically works with developers to refine it; other stakeholders or specialists may contribute when useful



AI in the Scrum Events – Support, Not Substitute

- **Backlog refinement** – AI can help draft or split stories and surface hidden risks, but the team still owns the decision
- **Sprint planning** – AI can summarise a story's likely complexity, yet planning poker exists to build **shared understanding**: don't outsource the estimate
- **Daily scrum** – AI can summarise commits and PRs, but the event's purpose is synchronisation, not status theatre
- **Review & retro** – AI can cluster tickets, PRs, and retro notes to cut prep time
- **Rule of thumb**: keep AI on the inputs and summaries; keep humans on the judgement and the conversation
- **Looking ahead** – when AI writes code, your **Definition of Done** must cover human review and understanding (**Week 4**)

Scrum FAQs

- **Do all of my sprints need to be the same length?**
 - Typically, **yes**: it leads to better predictability and 'rhythm'
- **Can the sprint goal be changed during a sprint?**
 - **No** – this is a promise made between the team and the Product Owner
 - If the **sprint goal becomes obsolete**, the Product Owner **may cancel the sprint**
 - However, **sprint backlog items can be updated / modified / refined** as needed to match the sprint goal
 - In particular, developers may adjust the work and collaborate with the Product Owner to **renegotiate scope and implementation details**, provided the sprint goal is **not endangered**

Scrum FAQs

- **Is the 2020 Scrum Guide still the definitive reference?**
 - Yes – the **2020 Scrum Guide** (Schwaber & Sutherland) remains the **official definition of scrum**, and it is what we teach
- **What is the Scrum Guide Expansion?**
 - An evolving companion to the 2020 Scrum Guide; the current version is **v2026.1 (January 2026)**
 - It adds guidance on product thinking, complexity, professional practice, AI, and modern software engineering
 - Treat it as further (optional) reading, not a replacement for the 2020 Scrum Guide – <https://scrumexpansion.org/>

Scrum FAQs

- **What happens if a user story isn't completed within a sprint?**
 - If it doesn't meet the Definition of Done (**Week 4**), it does not count towards velocity
 - The unfinished user story is reconsidered in the product backlog (it may be reordered, split, changed, or even dropped)
 - Partial story points are not counted
- **Should an unfinished user story be re-estimated?**
 - Scrum does not prescribe a rule
 - **Some may re-estimate it**, reflecting that it requires (likely) less effort to complete given that some of the tasks were completed. This is helpful when performing velocity-driven sprint planning
 - Others argue that you **shouldn't re-estimate it**, to preserve the original sizing and avoid distorting historical velocity (e.g. "steps" on the burndown chart)
 - Whatever approach is used, **apply it consistently** so that historical velocity remains interpretable

Scrum FAQs

- **Can I have a sprint purely for infrastructure design / set up?**
 - **Yes**, if the **work contributes towards the Product Goal** and produces a valuable, useful **increment**
 - Infrastructure work can include CI/CD pipelines, environment/deployment automation, observability, security controls, platform capabilities, etc.
 - The increment does **not** have to be a visible end-user feature
 - Avoid treating infrastructure as a separate phase that must happen before “real” product work
 - Terms such as “sprint zero” or “design sprint” exist in practice, but are not officially scrum events or special sprint types
 - With **AI accelerating feature and code generation**, enabling infrastructure, validation, automation, and guardrails may become even more important parts of product delivery

Scrum FAQs

- **How much work should we plan for our first sprint if AI can generate an entire system?**
 - **Don't** plan based on how much code AI can generate – generating code is only one **part of delivering a “Done” increment**
 - Your first sprint is about calibrating what a team can actually complete, including **understanding, reviewing, testing, debugging, integration, and documentation**
 - With no historical forecast, choose a **coherent Sprint Goal** and only as much work as you are reasonably confident you can finish
 - **You remain accountable for what AI produces** – in the Week 13 Q&A, you may be **asked to explain** your design, code, decisions, and how the system works
 - A fully working system that nobody understands is **not** a successful outcome

Scrum FAQs

- **How do multiple teams coordinate on a larger system?**

- Scrum does not prescribe how multiple teams must coordinate
- One approach is “**Scrum of Scrums**”, where team representatives meet regularly to coordinate and resolve cross-team impediments
- In practice, some organisations use **hybrid governance**, e.g. fixed programme milestones (more waterfall-like) with iterative scrum delivery inside teams

- **How does a scrum team involve an external vendor?**

- A **vendor may operate as a separate delivery team**, or participate as a stakeholder
- If close collaboration is needed, a vendor representative may attend relevant scrum events (e.g. sprint planning or review)
- Coordination often works best with **clearly defined interfaces and expectations**, supported by contracts that allow for iterative delivery
- **Prefer frequent integration** over hand-off points – shared APIs, CI/CD, integration testing, and common quality expectations help to reduce late “surprises”

Exercise & Lab

Exercise & Lab

- **Exercise:**

- Collate the user stories your group put together last week
- **Start estimating your product backlog** using the techniques discussed today
- ‘Groom’ and refine it according to the customer’s clarifications, and split any stories that are estimated to be too large

- **Lab:**

- Lab available in eLearn
- Learn how to store estimates in Jira
- Learn how to plan and start a sprint



Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 🖥️
- <http://smu.sg/is212-reflections-2026>

